

# Assembly Guide: Connecting the Dodgeball System

## 1. The Player (Dodger) Assembly

To make the player move and throw, we combine the input actions and scripts onto the box (player object).

- **Identity:** Select your Player box and assign the **Dodger** tag from the dropdown.
  - **Physics:** Ensure the **Rigidbody** has **Freeze Rotation** checked for X, Y, and Z.
  - **Brain:** On the **Player Input** (add the **Player Input** component if it does not exist for your player) component:
    - Ensure your **Input Action Asset** is in the "Actions" slot of the **Player Input** component.
    - Verify **Behavior** is set to **"Send Messages"**.
  - **Functionality:** Attach the **PlayerController** and **BallLauncher** scripts.
  - **The Launch Point:** Drag your **LaunchPoint** (the empty child object you may create to show where the ball would be) into the empty **"Launch Point"** slot on the **BallLauncher** script component.
- 

## 2. The Ball Assembly

The ball needs to be recognized by the scripts and behave with the physics we defined.

- **Identity:** Select the Sphere and assign the **Ball** tag.
  - **Physics:** Set the **Rigidbody** Collision Detection to **Continuous**.
  - **Bounce:** Find your **BouncyBall** Physic Material and drag it into the **Material** slot of the **Sphere Collider**.
  - **Logic:** Attach the **BallCollision** script directly to the sphere.
- 

## 3. The Environment & Walls

This ensures the ball bounces off boundaries and triggers a "Dodge" score.

- **Boundary Walls:** Select all your arena wall objects and assign the **Wall** tag. This prevents the "Tag not defined" error during collisions.
  - **Dodge Sensor (The Invisible Wall):**
    - Select the large cube behind the player.
    - Ensure **"Is Trigger"** is checked on its collider.
    - Attach the **DodgeSensor** script.
    - (Optional) Uncheck **Mesh Renderer** to make it invisible during gameplay.
-

#### 4. System Logic: How they "Talk" to each other

Once these assignments are made, the system functions as follows:

| Action           | Result                                                                                                                                 |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Press WASD       | The <b>Player Input</b> sends a message to <b>PlayerController</b> to move the Rigidbody.                                              |
| Press Space      | The <b>Player Input</b> sends a message to <b>BallLauncher</b> , which moves the <b>Ball</b> to the <b>LaunchPoint</b> and adds force. |
| Ball hits Player | The <b>BallCollision</b> script detects the <b>Dodger</b> tag and deactivates the player.                                              |
| Ball hits Sensor | The <b>DodgeSensor</b> script detects the <b>Ball</b> tag and logs a point.                                                            |

---

#### Pre-Play Checklist

1. **Names:** Does the Action Map name in your Input Asset match the name used in your **PlayerController** script?
2. **Tags:** Is the Player specifically tagged **Dodger** (capital D)?
3. **Slots:** Is there any "Missing" or "None" slot in your Inspector for the scripts? (Specifically the **LaunchPoint** on the Player).

# Guide d'assemblage : Connexion du système de Dodgeball

## 1. Assemblage du Joueur (Dodger)

Pour permettre au joueur de se déplacer et de lancer, nous combinons les actions d'entrée et les scripts sur le cube (l'objet joueur).

- **Identité** : Sélectionnez votre cube **Player** et assignez-lui le tag **Dodger** dans le menu déroulant.
- **Physique** : Assurez-vous que sur le **Rigidbody**, les cases **Freeze Rotation** sont cochées pour X, Y et Z.
- **Système d'Entrée (Input)** : Sur le composant **Player Input** (ajoutez-le s'il n'existe pas encore sur votre joueur) :
  - Assurez-vous que votre **Input Action Asset** est placé dans le slot "Actions".
  - Vérifiez que le **Behavior** est réglé sur **"Send Messages"**.
- **Fonctionnalité** : Attachez les scripts **PlayerController** et **BallLauncher**.
- **Le Point de Lancement (Launch Point)** : Faites glisser votre objet **LaunchPoint** (l'objet enfant vide que vous avez créé pour indiquer d'où part la balle) dans l'emplacement vide **"Launch Point"** du script **BallLauncher**.

## 2. Assemblage de la Balle

La balle doit être reconnue par les scripts et se comporter selon la physique que nous avons définie.

- **Identité** : Sélectionnez la Sphère et assignez-lui le tag **Ball**.
- **Physique** : Réglez la détection de collision (**Collision Detection**) du **Rigidbody** sur **Continuous**.
- **Rebond** : Trouvez votre matériau physique **BouncyBall** et faites-le glisser dans le slot **Material** du **Sphere Collider**.
- **Logique** : Attachez le script **BallCollision** directement sur la sphère.

## 3. Environnement et Murs

Cela garantit que la balle rebondit sur les limites et déclenche un point d'esquive.

- **Murs d'enceinte** : Sélectionnez tous les objets "murs" de votre arène et assignez-leur le tag **Wall**. Cela évite l'erreur "Tag not defined" lors des collisions.
- **Capteur d'Esquive (Le mur invisible)** :
  - Sélectionnez le grand cube situé derrière le joueur.
  - Assurez-vous que la case **"Is Trigger"** est cochée sur son Collider.
  - Attachez le script **DodgeSensor**.
  - (*Optionnel*) Décochez le **Mesh Renderer** pour le rendre invisible pendant le jeu.

## 4. Logique du système : Comment les objets "communiquent"

Une fois ces réglages effectués, le système fonctionne comme suit :

| Action                     | Résultat                                                                                                                                            |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Appuyer sur WASD/ZQSD      | Le <b>Player Input</b> envoie un message au <b>PlayerController</b> pour déplacer le Rigidbody.                                                     |
| Appuyer sur Espace         | Le <b>Player Input</b> envoie un message au <b>BallLauncher</b> , qui déplace la <b>Balle</b> vers le <b>LaunchPoint</b> et lui applique une force. |
| La balle touche le Joueur  | Le script <b>BallCollision</b> détecte le tag <b>Dodger</b> et désactive le joueur.                                                                 |
| La balle touche le Capteur | Le script <b>DodgeSensor</b> détecte le tag <b>Ball</b> et enregistre un point.                                                                     |

---

### Liste de vérification (Checklist) avant de lancer

- **Noms** : Le nom de la "Action Map" dans votre Asset d'Input correspond-il exactement à celui utilisé dans votre script **PlayerController** ?
- **Tags** : Le joueur possède-t-il bien le tag **Dodger** (avec un **D** majuscule) ?
- **Slots** : Y a-t-il des emplacements marqués **"Missing"** ou **"None"** dans l'Inspecteur de vos scripts ? (Vérifiez surtout le **LaunchPoint** sur le joueur).